

Phase 2: Smart Lead Qualification

Vyapar AI — Universal AI Employee Platform

Status: PLAN ONLY — NOT IMPLEMENTED

Date: June 2, 2026

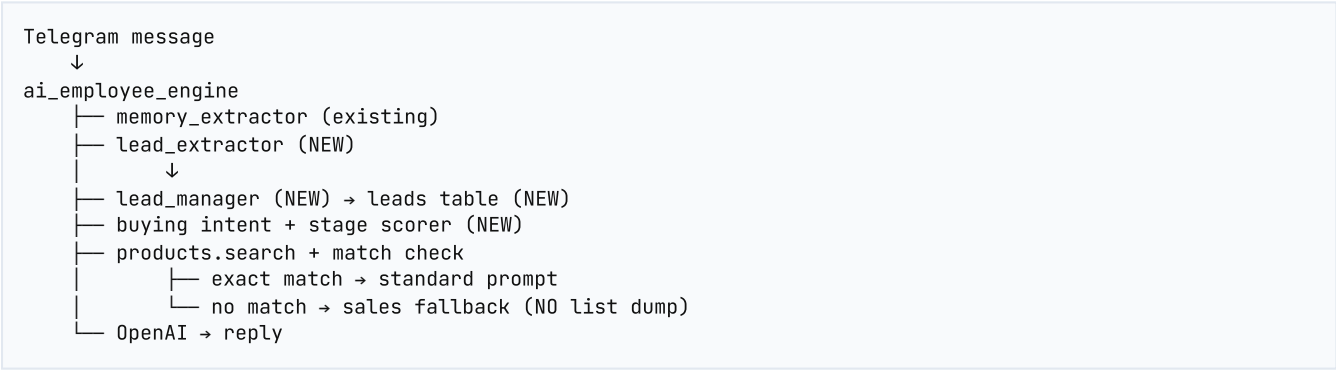
Builds on: Phase 1 (company_manager, COMPANY_ID, dynamic products)

Goal: Transform FAQ bot → sales employee with lead intelligence

1. Problem Statement

Today	Target
Answers FAQs via OpenAI + product injection	Detects buying intent and qualifies leads
Generic "sales" intent keyword	Nepali/English purchase phrases + signal scoring
User facts in user_memory only	Structured leads with stages in DB
No match → dumps full package list	Acknowledge → alternative → collect phone → save lead
"Be helpful" prompt tone	"Act like sales employee" — move toward conversion

2. Architecture Overview



Pipeline order (after Phase 2)

1. sanitize + save user turn (unchanged)
2. extract_memory_facts (unchanged)
3. extract_lead_signals (NEW)
4. detect_buying_intent (NEW)
5. upsert_lead via lead_manager (NEW)
6. direct memory answer (unchanged)
7. company config check (unchanged)
8. resolve sales_mode + product match (NEW)
9. compose prompt with lead_block + sales hint (NEW)
10. OpenAI + save reply (unchanged)

3. Files to Create (2)

3.1 lead_extractor.py

Function	Purpose
detect_buying_intent(text)	Nepali + English purchase/install patterns
extract_lead_signals(text)	budget, location, speed, urgency
extract_lead_fields(text, memory)	customer_name, location, budget, requested_speed, phone
score_lead_stage(fields, signals, buying)	new interested qualified hot

Buying intent patterns:

```
internet jodnu cha / jodna cha
package lina cha / linu cha
net chahiyo / internet chahiyo
connection chahiyo
install garna cha
subscribe / new connection / lagau / laguna
```

Lead signals:

Signal	Examples
Budget	NPR 12,000, bar hajar, budget, kati paisa
Location	City aliases + ma ... baschhu, area names
Speed	100 mbps, fast net, high speed
Urgency	aile, chhito, urgent, bholi, chada, asap

3.2 lead_manager.py

```
LEAD_STAGES = ("new", "interested", "qualified", "hot")

upsert_lead(user_id, company_id, fields, signals, stage, source_message)
get_active_lead(user_id, company_id)
update_lead_stage(lead_id, stage)
lead_to_prompt(lead)           # inject into system prompt
export_leads(company_id)       # admin CLI later
```

Upsert rules: One active open lead per (user_id, company_id). Never downgrade stage. Merge fields — fill empty only.

4. Files to Modify (6)

File	Change
memory_db.py	Add Lead SQLAlchemy model
ai_employee_engine.py	Wire lead pipeline + sales mode branch
intent_engine.py	buying_intent, lead_qualification intents + hints
prompts.py	SALES_EMPLOYEE_HINT, lead_block, sales_mode param
products.py	find_best_product_match(), has_exact_speed_match()
env.example	LEAD_ACTIVE_DAYS=30 (optional)

Unchanged: main.py, openai_engine.py, memory.py core, company_manager.py

5. Lead Stages — Scoring Rules

Stage	Criteria
new	buying_intent=True OR sales intent with no signals
interested	new + at least 1 signal (budget OR location OR speed OR urgency)
qualified	(name) + (phone) + (location OR speed)
hot	qualified + urgency OR explicit install/lina/jodnu + phone present

6. Database — leads Table

```
leads
├─ id                INTEGER PK
├─ user_id           STRING(64) INDEX
├─ company_id        STRING(64) INDEX
├─ stage             new | interested | qualified | hot
├─ customer_name     STRING(120)
├─ location          STRING(160)
├─ budget            STRING(64)
├─ requested_speed   STRING(64)
├─ phone             STRING(32)
├─ urgency           low | medium | high
├─ buying_intent     BOOLEAN
├─ matched_product   STRING(160)
├─ alternative_product STRING(160)
├─ source_message    TEXT
├─ notes             TEXT
├─ created_at, updated_at
```

7. Field Extraction Mapping

Lead field	Primary	Fallback
customer_name	lead_extractor patterns	user_memory.name
location	extract_city() + area regex	user_memory.city
budget	NPR/Rs/hajar regex	—
requested_speed	\d+ mbps patterns	user_memory.package_interest
phone	extract_phone()	user_memory.phone

8. Sales Mode — Unavailable Package

Triggers

- `detect_buying_intent(text) == True`
- sales/pricing intent + speed not in catalog
- `search_products()` empty but speed/buying mentioned
- Requested speed higher than max catalog product

Behavior when no exact match

Do NOT	DO
Dump full package list in prompt	Acknowledge customer need first
Repeat FAQ-style catalog	Propose one closest alternative
Ignore lead context	Ask for phone if missing
—	Create/update lead in <code>lead_manager</code>

Example reply (Nepali)

"500 Mbps ahile hamro catalog ma chhaina, tara 200 Mbps package le dherai ghar ra office ko lagi upayukta hunchha. Yadi tapai lai yo bhanda farak option chahiyo bhane hamro team le confirm garchha. Tapai ko contact number dinu hola, ma follow-up ko lagi save garidinchhu."

9. Prompt Changes (prompts.py)

```
def compose_system_prompt(
    business_block, memory_block, intent_block,
    knowledge_block, product_block,
    lead_block: str = "", # NEW
    sales_mode: bool = False, # NEW
) → str
```

SALES_EMPLOYEE_HINT:

- Act as sales employee, not FAQ bot
- Acknowledge specific need first
- If exact product unavailable — one alternative only, no full list
- Ask for phone naturally if missing and intent is strong
- Concise for Telegram; reply in user's language

10. Logging Events

```
LEAD_SIGNALS_DETECTED user_id=... signals={...}
LEAD_UPSERTED user_id=... company_id=... stage=qualified
SALES_MODE_ACTIVE user_id=... reason=no_exact_match
LEAD_PHONE_COLLECTED user_id=... stage=hot
```

11. Backward Compatibility

- Memory system untouched — lead reads from it
- FAQ/support messages — no lead unless buying intent or signals
- generate_employee_reply signature — optional params only
- All leads scoped by company_id (Phase 1)
- No admin panel — CLI export only

12. Testing Checklist

#	Input	Expected
1	Internet jodnu cha	buying_intent, lead stage≥new, sales tone
2	100 mbps package lina cha, Kathmandu ma	stage=interested, location saved
3	Mero naam Ram ho, 9800000000, 150 mbps chahiyo	stage=qualified or hot
4	500 mbps chahiyo (not in catalog)	No full list; alternative + phone ask
5	Support hours kati ho?	FAQ mode, no lead
6	Mero naam ke ho?	Direct memory, no lead side effect
7	Repeat user next day	Lead upserted, stage not downgraded
8	COMPANY_ID=hons	lead.company_id=hons

13. Risks & Mitigations

Risk	Mitigation
False-positive leads on "help"	Require buying_intent OR 2+ signals

Duplicate leads	Upsert on (user_id, company_id) 30-day window
AI still dumps packages	Sales mode omits full product_block
Regex misses Nepali variants	Iterative patterns + log unmatched messages
Render partial deploy	Deploy all files together

14. Out of Scope (Phase 2)

Admin dashboard · CRM webhook · Telegram admin alert on hot lead · Vector search · Background worker · Alembic · WhatsApp

15. Implementation Order (~5–7 days)

Day	Task
1	Lead model + lead_manager CRUD/upsert
2	lead_extractor — intent, signals, stage scorer
3	products.py — find_best_product_match()
4	prompts.py + intent_engine.py sales hints
5	ai_employee_engine.py full pipeline
6	Telegram testing + admin_export_leads.py
7	README_DEPLOY.md + Render deploy

Estimated: ~400–500 lines across 2 new + 6 modified files

16. Approval Checklist

1. One active lead per user per tenant (upsert, 30-day window)?
2. Stage scoring rules (\$5) acceptable?
3. Unavailable package → strict no list dump rule?
4. Phone collection as primary hot-stage goal?
5. admin_export_leads.py in Phase 2 or defer?

Reply **"approved"** with adjustments to begin implementation.